Chapitre 3:
Les modules

Plan



1. Les modules
    1.1. Définition
    1.2. Création
    1.3. Utilisation
1. Les packages
    2.1. Définition
    2.2. Exemples
3. Compléments

Les Modules : Définition

- Lorsque vous quittez et entrez à nouveau dans l'interpréteur Python, tout ce que vous avez déclaré dans la session précédente est perdu.

- Afin de rédiger des programmes plus longs, vous devez utiliser un éditeur de texte, préparer votre code dans un fichier et exécuter Python avec ce fichier en paramètre (création d'un script).

- Lorsque votre programme grandit, vous pouvez séparer votre code dans plusieurs fichiers. Ainsi, il vous est facile de réutiliser des fonctions écrites pour un programme dans un autre sans avoir à les copier.

Les Modules : Définition

- Python vous permet de placer des définitions dans un fichier et de les utiliser dans un script ou une session interactive.

- Un tel fichier est appelé un ***module*** et les définitions d'un module peuvent être importées dans un autre module ou dans le module *main* (qui est le module qui contient vos variables et définitions lors de l'exécution d'un script au niveau le plus haut ou en mode interactif).

- **Un module** est un fichier contenant des définitions et des instructions. Son nom de fichier est le nom du module suffixé de **.py**

Les Modules : Création

- Nous allons créer un fichier nommé puissance.py qui va définir deux fonctions carre() et cube(). Un tel fichier est appelé un *module* et il va pouvoir être *importé* dans un autre fichier, et en particulier dans le fichier qui contient le programme principal.

C:\Users\Sawssen\python codes\CoursModules\puissance.py

puissance.py

```
1   # -*- coding: utf-8 -*-
2   """
3   Created on Tue Apr 26 14:05:32 2022
4
5   @author: Sawssen
6   """
7
8   def carre(valeur):
9       resultat = valeur**2
10      return resultat
11
12  def cube(valeur):
13      resultat = valeur**3
14      return resultat
```

Les Modules : Exemple d'utilisation (1/7)

- Il est maintenant possible d'utiliser dans un programme principal les fonctions qui ont été définies dans le module **puissance.py**
- **Pour cela,** il faut importer les fonctions à partir du module.

puissance.py | test1.py

```
# -*- coding: utf-8 -*-
"""
Created on Tue Apr 26 14:14:24 2022

@author: Sawssen
"""
from puissance import carre
x=int(input('donnez un entier: '))
print('le carré de ', x , 'est ', carre(x))

```

```
In [59]: runfile('C:/Users/Sawssen/python codes/CoursModules/test1.py', wdir='C:/Users/Sa
python codes/CoursModules')
Reloaded modules: puissance

donnez un entier: 5
le carré de  5 est  25
```

N.B: le fichier puissance.py doit être dans le même répertoire que le programme principal.

Les Modules : Exemple d'utilisation (2/7)

- Il est maintenant possible d'utiliser dans un programme principal les fonctions qui ont été définies dans le module **puissance.py**
- **Pour cela,** il faut importer les fonctions à partir du module.

puissance.py | test1.py

```
1   # -*- coding: utf-8 -*-
2   """
3   Created on Tue Apr 26 14:14:24 2022
4
5   @author: Sawssen
6   """
7   from puissance import carre
8   x=int(input('donnez un entier: '))
9   print('le carré de ', x , 'est ', carre(x))
10  y=cube(x)
11  print('le cube de ', x , 'est ', y)
```

```
In [61]: runfile('C:/Users/Sawssen/python codes/CoursModules/test1.py', wdir='C:/Users/Sawssen/
python codes/CoursModules')
Reloaded modules: puissance

donnez un entier: 2
le carré de  2 est  4
Traceback (most recent call last):

  File "C:\Users\Sawssen\python codes\CoursModules\test1.py", line 10, in <module>
    y=cube(x)

NameError: name 'cube' is not defined
```

Remarque: dans cet exemple, nous avons importé uniquement la fonction carre().

Les Modules : Exemple d'utilisation (3/7)

- On importe explicitement toutes les fonctions

```
from puissance import carre, cube
x=int(input('donnez un entier: '))
print('le carré de ', x , 'est ', carre(x))
print('le cube de ', x , 'est ', cube(x))
```

```
donnez un entier: 2
le carré de  2 est  4
le cube de  2 est  8
```

Les Modules : Exemple d'utilisation (4/7)

- Maintenant, nous allons importer toutes les fonctions (avec *)

```
7   from puissance import *
8   x=int(input('donnez un entier: '))
9   print('le carré de ', x , 'est ', carre(x))
0   print('le cube de ', x , 'est ', cube(x))
1
```

```
In [65]: runfile('C:/Users/Sawssen/python codes/CoursModules/test1.py
python codes/CoursModules')
Reloaded modules: puissance

donnez un entier: 3
le carré de  3 est  9
le cube de  3 est  27
```

Les Modules : Exemple d'utilisation (5/7)

- On importe le module

```
import puissance
x=int(input('donnez un entier: '))
print('le carré de ', x , 'est ', puissance.carre(x))
print('le cube de ', x , 'est ', puissance.cube(x))
```

```
donnez un entier: 4
le carré de  4 est  16
le cube de  4 est  64
```

N.B:

Dans ce cas, il faut préciser le nom du module devant la fonction.

Les Modules : Exemple d'utilisation (6/7)

- On importe le module et on lui donne un alias

```
"""
import puissance as pu
x=int(input('donnez un entier: '))
print('le carré de ', x , 'est ', pu.carre(x))
print('le cube de ', x , 'est ', pu.cube(x))
```

```
donnez un entier: 5
le carré de  5 est  25
le cube de  5 est  125
```

Les Modules : Exemple d'utilisation (7/7)

- On importe une fonction d'un module et on lui donne un alias

```
from puissance import carre as ca
x=int(input('donnez un entier: '))
print('le carré de ', x , 'est ', ca(x))
```

```
donnez un entier: 3
le carré de  3 est  9
```

Importation de modules

Commande	Rôle	Exemple
import nom_module	Chargement d'un module	>>> import math
import nom_module as nom	Nommer un module	>>> import math as m >>> m.sqrt(2)
from nom_module import *	Chargement de l'intégralité du module	>>> from math import *
from nom_module import nom_fonction	Chargement d'une ou plusieurs fonctions d'un module	>>> from random import randint >>> randint(0,10) 7
dir(nom_module)	Affichages des fonctions du module	>>> dir(math)
help(nom_module.fonction) ou help(nom_module)	Aide sur une fonction ou un module	>>> help(m.ceil) >>> help(math)

Exemples de modules courants

- **math:** fonctions et constantes mathématiques de base (sin, cos, exp, pi...).
- **random :** génération de nombres aléatoires.
- **time :** permet d'accéder aux fonctions gérant le temps.
- **numpy :** module incontournable du calcul scientifique.
- **sys :** fournit les paramètres et fonctions liées à l'environnement d'exécution.
- **os :** fournit l'accès aux services génériques du système d'exploitation.
- **Tkinter :** interfaces graphiques

….

Les packages : Définition

- Quand on a un grand nombre de modules, il peut être intéressant de les organiser dans des dossiers.

- Un dossier qui rassemble des modules est appelé un **package** (un paquet en français).

- Le nom du package est le même que celui du dossier. Par exemple, on crée un dossier **package1** dans lequel on place le fichier **module1.py** contenant la fonction suivante:

```
def fonction1(a):
    return a**2
```

Les packages : Exemples

- On peut ensuite utiliser la fonction **fonction1()** définie dans **module1.py**, en important **package1.module1** comme dans l'exemple suivant :

```
import package1.module1

u = package1.module1.fonction1(3)
print("u vaut", u)
```

N.B: Il est aussi possible d'avoir des dossiers imbriqués, c'est-à-dire des dossiers qui contiennent d'autres dossiers.

Compléments

- A l'intérieur d'un module, le nom du module (en tant que chaîne de caractères) peut être obtenu grâce à la variable globale **__name__**

- Exemple: si on utilise le module **puissance.py** défini précédemment, on a dans l'interpréteur:

```
>>> import puissance
>>> puissance.__name__
'puissance'
```

Compléments

- Un même fichier **.py** peut jouer le rôle de script ou de module suivant son utilisation.

- Exécution d'un module en tant que script

- Quand on utilise un fichier en tant que *script*, la variable globale __name__ prend pour valeur « __main__ ». Ceci permet d'avoir dans le fichier un bloc d'instructions qui sera exécuté uniquement lorsque le fichier est lancé en tant que script.
- Par exemple, pour le fichier **puissance2.py** suivant :

```
def carre(valeur):
    resultat = valeur**2
    return resultat

if __name__ == "__main__":
    a = 3
    print("le carre vaut", carre(a))
```

On peut lancer une exécution directe en tant que script et on obtient alors :

```
le carre vaut 9
```

Compléments

- Ou bien on peut importer puissance2.py en tant que module, par exemple dans le fichier suivant :

```
import puissance2 as pu2

b = 4
print("le carre vaut", pu2.carre(b))
```

Si on exécute ce script, on obtient :

```
le carre vaut 16
```